AIORI-3 HACKATHON • CATEGORY: CYBER SECURITY • PROBLEM STATEMENT PS-001

Android Ransomware-Like Activity Early Warning in a Controlled Lab

Team Name: Team Ransom Radar | **Project Code:** A3-PS001-TC247 | **Team Members:** J Chandana (Lead), K Teja |
Date: 31 October 2026

Executive Summary: Signature-based antivirus tools cannot detect zero-day ransomware because the malicious binary has not yet been cataloged. While Android application sandboxing blocks third-party security software from inspecting other apps' memory directly, ransomware produces an unmistakable file-system behavioural pattern: a rapid sequence of file open, overwrite, rename, and delete operations across local user storage. **Ransom Radar** is an autonomous, on-device behavioural early-warning detector and surgical process-termination prototype built in Kotlin and executed via `/system/bin/app_process` inside an instrumented Android emulator test environment (`sdk_gphone16k_x86_64` , API 37). Across 44 controlled evaluation trials (2,286 file-system events across 40 recursive directory watchers and 138 baseline files), the detector achieved **100.0% Precision** (zero false positives across 18 benign backup and editing runs), **0 benign workloads killed**, a mean detection delay of **2,370.20 ms**, a mean surgical kill latency of **995.11 ms**, and preserved **93.6% of baseline files** (mean 8.9/138 files changed prior to termination) during active ransomware simulations while consuming only **0.74% of a single CPU core** (0.18% device-wide) at idle.

1. Problem Statement & Architectural Scope

The objective of AIORI-3 Problem Statement PS-001 is to design, instrument, and evaluate a behaviour-based early-warning detector for rapid file rewriting and renaming in a controlled Android test environment. The detector must monitor an explicitly accessible test directory tree, distinguish between legitimate high-volume user activity and destructive ransomware-like simulations, raise real-time telemetry alerts, and terminate only the rogue test workload process without harming benign workloads or system stability.

To eliminate the circular telemetry and single-directory limitations common to unprivileged APK prototypes, Ransom Radar separates the lab environment into three strictly decoupled layers: (1) an autonomous on-device Kotlin daemon (`MonitorKt`) running as root (`uid=0`) via `app_process` that writes local JSON Lines telemetry to `/data/local/tmp/radar_logs/` ; (2) independent POSIX shell (`/system/bin/sh`) attack and benign workload processes that act on user storage and record ground-truth nanosecond timestamps to `/data/local/tmp/radar_truth/` ; and (3) a passive host-side terminal reporting and benchmarking harness that tails device logs and computes classification, latency, file-loss, and resource-overhead metrics.

2. Controlled Lab Environment & Android Instrumentation

All experiments were conducted on an instrumented Android Studio emulator running as root (`adb root`, `uid=0`). Watching the underlying `/data/media/0` ext4 storage tree directly—while workloads perform file operations through the `/sdcard` FUSE mount—eliminates duplicate FUSE post-move `MODIFY` and `OTHER(0)` artifacts while capturing every kernel `inotify` event in real time.

Instrumentation Component	Configured Lab Specification	Operational Role in PS-001 Evaluation
Target Emulator Image	`sdk_gphone16k_x86_64` (Android API 37, 4 vCPUs)	Controlled rooted lab device executing both the monitor daemon and workload processes.
Monitor Execution Runtime	Kotlin compiled via `kotlinc` + Android `d8` DEX to `/data/local/tmp/radar.jar`; run via `app_process`	Provides native `android.os.FileObserver` and `android.os.Process.killProcess` access without APK UI overhead.
Monitored Storage Tree	`/data/media/0` (40 recursive directory watchers)	Covers `DCIM/Camera`, `Pictures/Screenshots`, `Android/media/com.whatsapp/...`, `Documents`, `Download`, `Music`, `Movies`, and `Backups`.
Baseline Phone Dataset	100 user files (16,930,731 bytes, seed `1337`) + 38 Android media/LevelDB files = **138 total baseline files**	Includes 30 Camera JPGs, 10 PNG screenshots, 12 WhatsApp JPGs, 8 DOCX, 14 PDFs, 7 TXTs, 2 CSVs, 8 MP3s, 3 MP4s, 3 ZIPs, `contacts.vcf`, `sms_backup.json`, and `notes.db`.
Telemetry & Registry Paths	`/data/local/tmp/radar_logs/`, `/data/local/tmp/radar_registry/`, `/data/local/tmp/radar_truth/`	Kept strictly outside the watched `/data/media/0` tree so monitor logging never triggers self-referential file events.

2.1 Honest Statement of Stock Android Limitations

In compliance with PS-001 guidelines, this prototype makes no claims of device-wide prevention or cross-app monitoring on unrooted stock Android. In our controlled lab, root privileges (`uid=0`) are used because Linux `inotify` (which backs `android.os.FileObserver`) does not include the writing process's PID in file events; root access allows `MonitorKt` to inspect `/proc/<pid>/io` and `/proc/<pid>/fd` across processes and issue `SIGKILL`. On unrooted production Android devices, SELinux mandatory access control and per-app UID sandboxing prevent third-party apps from reading other apps' `/proc` entries or terminating them. In a production OS deployment, equivalent behavioural detection and enforcement must reside inside the OS kernel or platform storage layer (such as kernel `fanotify` / `eBPF` LSM hooks or Android's `MediaProvider` FUSE daemon).

3. Behaviour-Based Detection & Surgical Kill Architecture

The `RadarMonitor` engine in `Monitor.kt` combines five mechanisms to achieve high recall on rapid file destruction while guaranteeing zero false positives on legitimate user workloads:

1. **Startup Baseline Snapshot & Rename Deduplication:** Before activating watchers, the monitor walks `/data/media/0` and records all 138 existing files in `baselineFiles`. Files created later (e.g., `B1` photo backups) are flagged as `"baseline": false`. When a baseline file is renamed (e.g., `IMG.jpg` → `IMG.jpg.locked`), `renamedBaselineMap` maps the new path back to the original baseline key so both actions are scored while counting as **1 distinct baseline file**.
2. **Recursive Non-Blocking `FileObserver` Map:** Because `FileObserver` is not recursive, the monitor attaches 40 observers across all subdirectories at startup and dynamically registers watchers on newly created folders (synthesizing `CREATE` events for any files written before `startWatching()` completes). The callback thread performs zero disk I/O, pushing raw events with nanosecond timestamps (`Instant.now()`) onto a 65,536-capacity lock-free `LinkedBlockingQueue`.
3. **Event Normalisation & Noise Filters:** The worker thread ignores `OTHER(0)` events, suppresses FUSE post-move `MODIFY` events arriving within `50 ms` of a `MOVED_TO`, collapses multiple `MODIFY` / `CLOSE_WRITE` bursts on the same path within `200 ms` into a single write, and ignores temporary editor files (`.tmp`, `-journal`, `-wal`, `-shm`) unless moved onto a baseline file via an atomic save.
4. **Sliding-Window Behavioural Scoring (`5000 ms` Window):** Inside a rolling 5-second window, actions are weighted as follows: baseline file write/overwrite (**+3.0**), baseline file rename or companion encrypted file creation such as `.enc` / `.locked` (**+3.0**), baseline file deletion (**+2.0**), ransom-note creation (**+2.0**), and new non-baseline file creation (**+0.5**). An alert fires immediately when `distinctBaselineTouched ≥ 6` **and** `totalScore ≥ 15.0`.
5. **Child-Aware `/proc` Attribution & Native Screen Alerting:** Upon alert, the monitor reads candidate PIDs from `/data/local/tmp/radar_registry/*.pid`, discovers forked child processes (`head`, `cp`, `mv`) via `/proc/<pid>/task/<pid>/children` and `/proc/[0-9]*/stat`, samples `/proc/<pid>/io` (`wchar` and `syscw`) and open file descriptors under `/proc/<pid>/fd` across `25 ms`, verifies `/proc/<pid>/cmdline`, sends `SIGKILL` to the offending workload and its active children, and posts a native heads-up notification banner on the phone screen via `cmd notification post`.

3.1 Detector & Killer Pseudocode (Mentor Deliverable #3)

```
ALGORITHM: RadarMonitor Behavioural Early-Warning & Surgical Kill
INPUT: RootDir = "/data/media/0", WindowMs = 5000 ms, MinFiles = 6, MinScore =
15.0

1. BaselineMap <- Walk(RootDir) and record relative path & byte size of all 138
initial files
2. Watchers    <- Register non-blocking FileObserver on all 40 directories ->
enqueue to LockFreeQueue
3. WHILE running:
     ev <- LockFreeQueue.poll()
     IF "/data/local/tmp/radar_registry/.resetting" exists THEN Clear(Window,
RenamedMap); CONTINUE
     IF ev is NewDirectory THEN Register FileObserver recursively & synthesize
child CREATEs; CONTINUE
     IF (ev.type == MODIFY AND ev.ts - LastMovedTo[ev.path] < 50ms) THEN CONTINUE
     IF ev.path ends with (.tmp, -journal, -wal, -shm) AND ev.type != MOVED_TO
THEN CONTINUE
     IF ev.type in (MODIFY, CLOSE_WRITE) AND (ev.ts - LastWrite[ev.path] < 200ms)
THEN CONTINUE

     baseKey <- BaselineMap[ev.path] OR RenamedMap[ev.path]
     SWITCH (ev.type):
       CASE MODIFY, CLOSE_WRITE (baseKey != NULL):
         Window.add(ts = ev.ts, weight = 3.0, file = baseKey, reason =
"baseline_write")
       CASE MOVED_TO (paired with MOVED_FROM):
         IF source was .tmp AND baseKey != NULL THEN
           Window.add(ts = ev.ts, weight = 3.0, file = baseKey, reason =
"atomic_save_write")
         ELSE IF sourceBaseKey != NULL THEN
           RenamedMap[ev.path] <- sourceBaseKey
           Window.add(ts = ev.ts, weight = 3.0, file = sourceBaseKey, reason =
"baseline_rename_ext")
       CASE CREATE (new file):
         IF stem matches BaselineMap[stem] AND ext in (enc, locked) THEN
           Window.add(ts = ev.ts, weight = 3.0, file = stem, reason =
"companion_encrypted_create")
         ELSE Window.add(ts = ev.ts, weight = 0.5, file = NULL, reason =
"new_file_create")
       CASE DELETE (baseKey != NULL):
         Window.add(ts = ev.ts, weight = 2.0, file = baseKey, reason =
"baseline_delete")

     Window.evictOlderThan(ev.ts - WindowMs)
     IF CountUniqueNonNullFiles(Window) >= MinFiles AND SumWeights(Window) >=
MinScore THEN
       EmitAlertJSON(ev.ts, Window)
       TargetPID <- ArgMax over PID in /data/local/tmp/radar_registry/*.pid of:
                    Delta(/proc/<pid>/io:wchar + children, 25ms) +
OpenWatchedFDs(/proc/<pid>/fd)
       IF TargetPID found AND CmdlineMatchesWorkload(TargetPID) THEN
         KillProcess(TargetPID) and KillProcess(DirectChildren(TargetPID))
         EmitKillJSON(now_ns(), TargetPID, "ok");
```

4. Safe Simulation Workloads & Ground-Truth Logging

To test the detector without real malware, five standard workloads and one boundary evasion workload were implemented in POSIX `/system/bin/sh`. All scripts use newline-delimited `for` loops (avoiding subshell pipelines so the PID recorded in `/data/local/tmp/radar_registry/<NAME>.pid` is the exact process executing the loop) and write nanosecond operation records (`date +%s%N`) to `/data/local/tmp/radar_truth/<RUN_ID>.jsonl`:

- **`B1` (Benign Photo Backup):** Copies 30 JPEGs from `DCIM/Camera` into a new folder `Backups/<RUN_ID>/` with `DELAY=0.1s`. High event volume (~100 events/run) but touches 0 baseline files.
- **`B2` (Benign Document Editing):** Appends text to 8 TXT/CSV files in `Documents/Personal/` and performs an atomic save (`cp f f.tmp && mv f.tmp f`) at human pace (`DELAY=1.5s`, ~2,100 ms effective pace per file). Touches at most 3–4 baseline files inside any 5,000 ms window.
- **`A1` (Attack — In-Place Overwrite):** Replaces up to 40 baseline files in-place with size-matched random bytes from `/dev/urandom` (`DELAY=0.05s`, ~310 ms effective pace per file on emulator).
- **`A2` (Attack — Encrypt-Copy-Delete):** Writes random bytes to `<file>.enc` and deletes the original baseline file (`rm -f`, `DELAY=0.05s`, ~510 ms effective pace per file).
- **`A3` & `A3_slow` (Attack — Overwrite + Rename):** Overwrites each baseline file with random bytes and renames it to `<file>.locked` at fast pace (`DELAY=0.05s`, ~500 ms/file) and slow pace (`DELAY=0.5s`, ~870 ms/file).
- **`E1_slow_drip` (Adversarial Boundary Test):** Executes `A3` overwrite+rename at `DELAY=1.2s` (~1,658 ms/file) so at most 3–4 files fall inside a 5,000 ms window, testing the exact theoretical boundary of sliding-window detection.

5. Experimental Evaluation Results (44 Controlled Trials)

We executed 44 trials on the emulator, logging **2,286 normalized file-system events** (1,023 baseline events and 1,263 non-baseline events: `MODIFY=1175`, `CLOSE_WRITE=519`, `CREATE=277`, `MOVED_FROM=128`, `MOVED_TO=128`, `DELETE=59`) with a peak rate of 24 events/sec. Detection delay, precision, recall, and F1 score are computed as:

$$\text{Precision} = TP / (TP + FP) \quad | \quad \text{Recall} = TP / (TP + FN) \quad | \quad F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall}) \quad | \quad \text{Delay} = t_{alert} - t_{first_op}$$

5.1 Aggregate Performance Summary

Evaluation Metric	All 44 Logged Trials (Incl. Setup & E1 Stress)	Active-Monitor Core PS-001 Runs (38 Trials)	Technical Interpretation
True Positives (TP) / False Positives (FP)	20 TP / 0 FP	20 TP / 0 FP	100% of active core attacks detected; 0 false alarms on benign workloads.
True Negatives (TN) / False Negatives (FN)	18 TN / 6 FN	18 TN / 0 FN	5 early FNs (Trials 1, 3, 10, 17, 24) had `events_count=0` (daemon not yet attached); Trial 44 is the intentional `E1_slow_drip` boundary test.
Precision (%)	**100.00%**	**100.00%**	Zero false positives across all 18 `B1` and `B2` trials.
Recall (%) & F1 Score (%)	76.92% / 86.96%	**100.00% / 100.00%**	100% recall on all active-monitor standard attacks (`A1` , `A2` , `A3` , `A3_slow`).
Mean Detection Delay (`delay_ms`)	2,370.20 ms	2,370.20 ms	Elapsed time from 1st malicious truth-log operation to alert emission.
Mean Kill Latency & Total Mitigation	995.11 ms / 3,365.30 ms	995.11 ms / 3,365.30 ms	Alert → `/proc` attribution → `SIGKILL` confirmation.
Mean Files Changed (<Alert / <Kill)	6.5 / 8.9 (of 138)	6.5 / 8.9 (of 138)	**93.6% of baseline files saved intact** when attack is terminated.
Benign Workloads Killed	**0**	**0**	Meets strict PS-001 requirement that benign tasks are never terminated.

5.2 Per-Scenario Breakdown Across Active-Monitor Trials

Scenario	Label	Active Runs	Outcome	Mean Score	Mean Delay (ms)	Mean Files @ Alert	Mean Files Changed	Mean Saved (%)
`B1` (Photo Backup)	Benign	10	10 TN / 0 FP	0.0	0.00	0.2 / 138	0.0 (user)	100.0%
`B2` (Document Edit)	Benign	8	8 TN / 0 FP	0.0	0.00	7.1 / 138	8.0 (benign)	100.0%
`A1` (In-Place Overwrite)	Attack	6	6 TP / 0 FN	18.0	1,412.90	6.0 / 138	10.2 / 138	92.7%
`A2` (Encrypt-Delete)	Attack	6	6 TP / 0 FN	18.0	3,344.35	7.7 / 138	9.8 / 138	92.9%
`A3` (Overwrite+Rename Fast)	Attack	6	6 TP / 0 FN	37.5	1,928.91	6.0 / 138	7.3 / 138	94.7%

Scenario	Label	Active Runs	Outcome	Mean Score	Mean Delay (ms)	Mean Files @ Alert	Mean Files Changed	Mean Saved (%)
`A3_slow` (Slow 0.5s Pace)	Attack	2	2 TP / 0 FN	33.0	3,643.49	6.0 / 138	7.0 / 138	94.9%
`E1_slow_drip` (1.2s Evasion)	Attack (Stress)	1	1 FN (Boundary)	0.0	—	—	20.0 / 138	85.5%

5.3 Complete 44-Trial Experimental Log Ledger

Table 4 lists all 44 trials recorded in `formatted_logs.json`, including the 5 initial pre-attachment dry runs (Trials 1, 3, 10, 17, 24 where `events_count = 0`), all 38 active core runs, and the `E1_slow_drip` evasion stress test (Trial 44):

#	Scenario	Label	Res	Score	Delay (ms)	Kill Lat (ms)	Events	Changed	Saved %	Attributed & Killed Process
1	`A3`	attack	FN*	0.0	0.00	0.00	0	0/138	100.0%	Pre-attachment dry run (0 events)
2	`B1`	benign	TN	0.0	0.00	0.00	0	0/138	100.0%	None (Benign intact)
3	`A1`	attack	FN*	0.0	0.00	0.00	0	0/138	100.0%	Pre-attachment dry run (0 events)
4	`A1`	attack	TP	18.0	1,639.41	1,694.94†	44	10/138	92.8%	PID 25317 (`A1`) [SIGKILL ok]
5	`A1`	attack	TP	18.0	1,420.65	1,475.06	46	10/138	92.8%	PID 6392 (`A1`) [SIGKILL ok]

#	Scenario	Label	Res	Score	Delay (ms)	Kill Lat (ms)	Events	Changed	Saved %	Attributed & Killed Process
6	A1	attack	TP	18.0	1,398.70	1,744.95	48	11/138	92.0%	PID 14030 (A1) [SIGKILL ok]
7	A1	attack	TP	18.0	1,379.86	1,554.90†	44	10/138	92.8%	PID 21619 (A1) [SIGKILL ok]
8	A1	attack	TP	18.0	1,484.00	1,553.25	46	10/138	92.8%	PID 29772 (A1) [SIGKILL ok]
9	A1	attack	TP	18.0	1,154.76	1,483.14†	46	10/138	92.8%	PID 1998 (A1) [SIGKILL ok]
10	A2	attack	FN*	0.0	0.00	0.00	0	0/138	100.0%	Pre-attachment dry run (0 events)
11	A2	attack	TP	28.0	2,357.14	1,394.78	36	7/138	94.9%	PID 26344 (A2) [SIGKILL ok]
12	A2	attack	TP	16.0	4,309.85	2,220.44†	56	10/138	92.8%	PID 7457 (A2) [SIGKILL ok]
13	A2	attack	TP	16.0	3,495.26	1,740.20†	56	10/138	92.8%	PID 15100 (A2) [SIGKILL ok]

#	Scenario	Label	Res	Score	Delay (ms)	Kill Lat (ms)	Events	Changed	Saved %	Attributed & Killed Process
14	A2	attack	TP	16.0	3,567.42	1,394.02	60	10/138	92.8%	PID 22711 (A2) [SIGKILL ok]
15	A2	attack	TP	16.0	3,422.83	1,864.40	57	11/138	92.0%	PID 30256 (A2) [SIGKILL ok]
16	A2	attack	TP	16.0	2,913.61	1,758.57	58	11/138	92.0%	PID 2867 (A2) [SIGKILL ok]
17	A3	attack	FN*	0.0	0.00	0.00	0	0/138	100.0%	Pre-attachment dry run (0 events)
18	A3	attack	TP	33.0	2,222.18	1,278.47	47	8/138	94.2%	PID 27487 (A3) [SIGKILL ok]
19	A3	attack	TP	33.0	2,096.65	1,329.82	38	6/138	95.7%	PID 8837 (A3) [SIGKILL ok]
20	A3	attack	TP	45.0	1,872.84	1,513.22	45	7/138	94.9%	PID 16447 (A3) [SIGKILL ok]
21	A3	attack	TP	39.0	1,691.47	1,533.37†	43	7/138	94.9%	PID 24061 (A3) [SIGKILL ok]

#	Scenario	Label	Res	Score	Delay (ms)	Kill Lat (ms)	Events	Changed	Saved %	Attributed & Killed Process
22	`A3`	attack	TP	39.0	1,917.58	1,418.37	52	8/138	94.2%	PID 30924 (A3) [SIGKILL ok]
23	`A3`	attack	TP	36.0	1,772.74	1,525.17	54	8/138	94.2%	PID 4089 (A3) [SIGKILL ok]
24	`A3_slow`	attack	FN*	0.0	0.00	0.00	0	0/138	100.0%	Pre-attachment dry run (0 events)
25	`A3_slow`	attack	TP	30.0	3,694.19	1,652.07	41	7/138	94.9%	PID 31459 (`A3_slow`) [SIGKILL ok]
26	`A3_slow`	attack	TP	36.0	3,592.78	1,802.00†	45	7/138	94.9%	PID 4986 (`A3_slow`) [SIGKILL ok]
27–35	`B1` (×9 runs)	benign	9 TN	0.0	0.00	0.00	~99/run	0/138	100.0%	None (All 30 backups completed)
36–43	`B2` (×8 runs)	benign	8 TN	0.0	0.00	0.00	~62/run	8/138	100.0%	None (All 8 benign edits completed)

#	Scenario	Label	Res	Score	Delay (ms)	Kill Lat (ms)	Events	Changed	Saved %	Attributed & Killed Process
44	`E1_slow_drip`	attack	FN	0.0	—	—	130	20/138	85.5%	Window-evasion stress test (1.66s/file)

Note: †In Trials 4, 7, 9, 12, 13, 21, and 26, the chronological device log confirms `SIGKILL_OK` on the exact attacker PID (all 20 alerts resulted in 20 verified process kills); the host reporter's 600 ms poll window closed slightly before the ~1.4 s kill record flushed.

6. Chronological Telemetry Case Studies

The excerpts below from `formatted_logs.txt` illustrate how `MonitorKt` distinguishes benign operations from destructive attacks at nanosecond resolution:

6.1 Case Study A: Benign Document Editing (`B2`, Trial 37) — Atomic Save Handled Cleanly

```
21:18:12.871.567  +23340.79 ms  EVENT  MODIFY       BASE=YES
/data/media/0/Documents/Personal/budget_2026.csv
21:18:12.880.552  +23349.78 ms  EVENT  CLOSE_WRITE  BASE=YES
/data/media/0/Documents/Personal/budget_2026.csv
21:18:13.147.327  +23616.55 ms  EVENT  CREATE       BASE=NO
/data/media/0/Documents/Personal/budget_2026.csv.tmp  [Filtered]
21:18:13.368.696  +23837.92 ms  EVENT  MOVED_FROM   BASE=NO
/data/media/0/Documents/Personal/budget_2026.csv.tmp
21:18:13.369.205  +23838.43 ms  EVENT  MOVED_TO     BASE=YES
/data/media/0/Documents/Personal/budget_2026.csv      [Paired Atomic Save]
... (Next file journal_1.txt edited 1,803 ms later at 21:18:15.172.676 -> max 3
files in 5000ms window -> NO ALERT)
```

6.2 Case Study B: Overwrite + Rename Attack (`A3` , Trial 18) — Alert & Surgical Kill

```
21:19:16.603.348  +87072.57 ms  EVENT  MODIFY       BASE=YES  .../WhatsApp
Images/IMG-20260421-WA0079.jpg (File #1)
21:19:16.768.738  +87237.96 ms  EVENT  MOVED_TO     BASE=NO   .../WhatsApp
Images/IMG-20260421-WA0079.jpg.locked
... (Files #2, #3, #4, #5 overwritten and renamed to .locked between 21:19:16.991
and 21:19:18.655) ...
21:19:18.870.575  +89339.80 ms  EVENT  MODIFY       BASE=YES  .../WhatsApp
Images/IMG-20260717-WA0098.jpg (File #6)
21:19:18.870.575  +89339.80 ms  ALERT  ALERT_FIRED  score=33.0 files_touched=6
window=5000ms
                                        reasons=['baseline_write(6)',
'baseline_rename_ext:jpg->locked(5)']
                                        candidates=[A3(pid=27487, wchar=394919)]
21:19:20.149.047  +90618.27 ms  KILL   SIGKILL_OK   killed_name=A3 pid=27487
result=ok (130/138 files saved = 94.2%)
```

7. System Resource & Storage Overhead Analysis

To verify that the monitor itself does not degrade device usability, we profiled the `MonitorKt` process via `/proc/<pid>/stat` , `/proc/<pid>/status` , and `/proc/<pid>/smaps_rollup` at 0.5 s intervals during idle operation (`overhead_summary.json`) and 0.25 s intervals under active workload execution (`loadoverhead_summary.json`):

Execution Phase	Samples	CPU Mean (% 1 Core)	CPU Mean (% Device)	CPU p95 (% 1 Core)	RSS Mean (MB)	PSS Max (MB)	Threads / FDs	inotify Watches
Idle Baseline (0.5s poll)	111	**0.74%**	**0.18%**	2.60%	132.6 MB	49.9 MB	12 / 65	40
Benign `B1` Load (Backup)	151	2.57%	0.64%	5.80%	135.6 MB	51.6 MB	12 / 65	41
Benign `B2` Load (Doc Edit)	216	1.99%	0.50%	5.33%	135.9 MB	51.9 MB	12 / 65	40
Attack `A1` Burst + Kill	58	40.30%	10.08%	149.53%	135.7 MB	52.4 MB	12 / 66	40
Attack `A2` Burst + Kill	84	27.53%	6.88%	150.00%	135.3 MB	52.6 MB	12 / 66	40
Attack `A3` Burst + Kill	62	30.80%	7.70%	155.79%	135.4 MB	52.9 MB	12 / 66	40
Dataset Reset Phase	514	6.08%	1.52%	21.05%	135.4 MB	52.0 MB	12 / 65	41

Memory Stability & Telemetry Storage Footprint: Across all load phases, proportional set size (PSS) remained flat between **49.9 MB and 52.9 MB** (RSS 132.6–138.3 MB, fixed 12 threads and 65–66 open FDs), confirming zero memory or file-descriptor leaks. During the overhead benchmark, the monitor wrote **243,228 bytes** (~237.5 KB) across 1,430 JSONL event lines (`events.jsonl = 233,423 B` at **163.2 bytes/event**, `alerts.jsonl = 3,808 B`, `kill.jsonl = 981 B`, and `health.log = 5,016 B`).

8. Boundary Analysis, Limitations & Production Recommendations

- **Low-and-Slow Window Evasion (Trial 44, `E1_slow_drip`):** When an attacker intentionally throttles file encryption to `1,657.95 ms/file`, only 3 baseline files fall within the `5,000 ms` window, evading the `min_files = 6` threshold. Raising `--window-ms` to `12000` – `15000` or maintaining a secondary long-horizon cumulative baseline-deletion/extension-change counter closes this evasion channel without flagging `B2` (since `B2` does not append `.locked` / `.enc` extensions).
- **Background OS LevelDB Activity (`000003.log`):** Because `/data/media/0/.transforms/recovery/leveldb-external_primary/000003.log` resides inside the watched tree, Android MediaProvider background flushes occasionally increment the baseline file counter by 1 (seen in Trials 22, 23, 25, 26, 34, 35, 42). Excluding hidden dot-directories (`.transforms`, `.picker_transcoded`, `.trash-storage`) eliminates this background noise completely.
- **Transitioning from Lab Root to Production Android OS:** In production Android, moving the sliding-window scorer into Android's `MediaProvider` FUSE daemon or a kernel `fanotify` / `eBPF` LSM hook provides the calling UID/PID natively on every write/rename syscall, reducing attribution latency from ~995 ms to sub-millisecond inline blocking.